module 1

Module 1 — Instrument Architecture

Virtual Harmonium Lab · ESP32 Digital Musical Workstation

Engineering Reference Document

Source: Polyphone 2.x Source Code (© 2013–2024 Davy Triponney, GPL v3)
Date compiled: 2026-07-27
Status: Authoritative · Do not modify without re-verifying source

---

> This document answers one question: **"What information defines an instrument?"**
> It does NOT cover playback, DSP, mixing, or audio rendering. Those belong to Module 2.

Table of Contents

1. Overall Instrument Hierarchy
2. SF2 Internal Structure
3. SFZ Internal Structure
4. Sample Representation
5. Region / Division Representation
6. Group / Global Division Representation
7. Instrument Parameters — Complete Table
8. Runtime vs Editor Parameters
9. Data Structures — Class Reference
10. Source Code Extraction Reference
11. Relationship Diagram
12. Embedded Notes Summary

---

1. Overall Instrument Hierarchy

What Polyphone Actually Uses

Polyphone does **not** have a concept called "group" separate from "instrument". The real hierarchy in Polyphone's data model is:

```
Soundfonts (root container)
  └── Soundfont (one open .sf2 or .sfz file)
        ├── Samples          (elementSmpl)
        ├── Instruments      (elementInst)
        │     ├── Global Division   (elementInst - special bag[0])
        │     └── Divisions         (elementInstSmpl - one per linked
sample)
        │           └── Modulators  (elementInstSmplMod)
        └── Presets          (elementPrst)
              ├── Global Division   (elementPrst - special bag[0])
              └── Divisions         (elementPrstInst - one per linked
instrument)
                    └── Modulators  (elementPrstInstMod)
```

Why Each Level Exists

Level	Class in Polyphone	SF2 Name	Why it exists
Soundfonts	`Soundfonts`	—	Root container for all open sf2 files
Soundfont	`Soundfont`	SF2 File	One .sf2 file. Holds metadata (INAM, ICRD...), lists of samples, instruments, presets
Sample	`Smpl`	SHDR	The raw audio data and its playback properties (rate, loop, root key, tuning)
Instrument	`InstPrst`	INST	Maps samples to key/velocity zones. Defines the per-zone parameters that modify how a sample sounds
Global Division	`Division`	ibag[0] (no sampleID generator)	Default parameters that apply to all zones within this instrument unless a zone overrides them

Level	Class in Polyphone	SF2 Name	Why it exists
Division (zone)	`Division`	ibag[n] (has sampleID generator)	A specific key+velocity range mapping one sample, with its own parameter overrides
Preset	`InstPrst`	PHDR	The MIDI-visible entry (bank + program number). Maps instruments to key/velocity zones. Adds one more layer of parameter offsets
Preset Division	`Division`	pbag[n] (has `champ_instrument` generator)	Links one instrument into a preset zone, with optional parameter offsets on top of the instrument

Critical Design Point

Polyphone uses the **same C++ class** (`InstPrst`) for both Instruments and Presets, and the **same class** (`Division`) for both instrument zones and preset zones. The distinction is encoded in the `ElementType` enum stored in `EltID` .

Source: `sources/core/model/instprst.h` , `sources/core/model/division.h`

---

2. SF2 Internal Structure

How an SF2 File is Organized on Disk

A SoundFont 2 file is a RIFF chunk with three sub-parts:

```
RIFF 'sfbk'
  LIST 'INFO'   -- metadata (INAM, ICRD, ICOP, etc.)
  LIST 'sdta'   -- sample audio data
    smpl        -- 16-bit PCM samples
    sm24        -- optional extra byte for 24-bit samples
  LIST 'pdta'   -- preset/instrument/sample descriptors
    phdr[]      -- preset headers
    pbag[]      -- preset bags (zones)
    pmod[]      -- preset modulators
```

```
    pgen[]      -- preset generators
    inst[]      -- instrument headers
    ibag[]      -- instrument bags (zones)
    imod[]      -- instrument modulators
    igen[]      -- instrument generators
    shdr[]      -- sample headers
```

Source: `sources/core/input/sf/sf2header.h`, `sources/core/input/sf/sf2sdtapart.h`, `sources/core/input/sf/sf2pdtapart.h`

How Polyphone Parses This Into Its Internal Model

The entry point is `InputParserSf::processInternal()` in `sources/core/input/sf/inputparsersf.cpp`.

It calls `parse()` which deserializes three objects:

```
Sf2Header header;
Sf2SdtaPart sdtaPart;
Sf2PdtaPart pdtaPart;
stream >> header >> sdtaPart >> pdtaPart;
```

Then `fillSf2()` populates the `SoundfontManager` singleton.

Step 1: Metadata

```
// From inputparsersf.cpp, lines 105-119
_sm->set(id, champ_ISNG, header.getInfo("isng"));
_sm->set(id, champ_name, header.getInfo("INAM"));
_sm->set(id, champ_IROM, header.getInfo("irom"));
_sm->set(id, champ_ICRD, header.getInfo("ICRD"));
_sm->set(id, champ_IENG, header.getInfo("IENG"));
_sm->set(id, champ_IPRD, header.getInfo("IPRD"));
_sm->set(id, champ_ICOP, header.getInfo("ICOP"));
_sm->set(id, champ_ICMT, header.getInfo("ICMT"));
_sm->set(id, champ_ISFT, header.getInfo("ISFT"));
```

All metadata is editor-only. None of this affects sound.

Step 2: Samples (SHDR records)

Each `Sf2PdtaPart_shdr` struct directly maps to an on-disk sample header:

```
// From sf2pdtapart_shdr.h
class Sf2PdtaPart_shdr {
```

```
    QString       _name;           // sample name (max 20 chars)
    quint32Reversed _start;        // start offset in smpl chunk (in
samples)
    quint32Reversed _end;          // end offset in smpl chunk
    quint32Reversed _startLoop;    // loop start (absolute, converted
to relative)
    quint32Reversed _endLoop;      // loop end (absolute, converted to
relative)
    quint32Reversed _sampleRate;   // sample rate in Hz
    quint8        _originalPitch;  // MIDI root note (0-127)
    qint8         _correction;     // fine pitch correction (-99 to +99
cents)
    quint16Reversed _wSampleLink;  // index of linked (stereo partner)
sample
    quint16Reversed _sfSampleType; // mono/left/right/linked flags
};
```

Polyphone converts absolute sample offsets to relative:

```
// inputparsersf.cpp line 223-226
value.dwValue = SHDR._startLoop.value - SHDR._start.value; // absolute
→ relative
sound->set(champ_dwStartLoop, value);
value.dwValue = SHDR._endLoop.value - SHDR._start.value;
sound->set(champ_dwEndLoop, value);
```

Step 3: Instruments and Their Bags (Zones)

The SF2 "bag" system is an indirection table:

```
inst[i]._iBagIndex → ibag[j] → {_modIndex, _genIndex}
                     ibag[j+1] → ...
```

Polyphone reads this and classifies each bag as **global** or a **sample-linked zone**:

```
// inputparsersf.cpp lines 274-296
global = 1;
for (int k = genmin; k < genmax; k++) {
    if (pdtaPart._igens[k]._sfGenOper.value == champ_sampleID)
        global = 0;    // This bag has a sampleID generator → it's a
zone
}

if (global)
```

```
    id2.typeElement = elementInst;      // parameters go to the global
division
else {
    id2.typeElement = elementInstSmpl; // new zone is created
    id2.indexElt2 = _sm→add(id2);
}
```

Generators from each bag are loaded as `AttributeType` key-value pairs into the corresponding `Division`.

Step 4: Presets and Their Bags

Identical mechanism, but the distinguishing generator is `champ_instrument` (generator #41) instead of `champ_sampleID` (#53):

```
// inputparsersf.cpp lines 403-413
for (int k = genmin; k < genmax; k++) {
    if (pdtaPart._pgens[k]._sfGenOper.value == champ_instrument)
        global = 0;     // This bag references an instrument
}
```

Bags and Zones — Explained Simply

SF2 Concept	Polyphone Concept	Class
`shdr[i]`	Sample metadata	`Smpl`
`inst[i]`	Instrument header	`InstPrst`
`ibag[0]` (no sampleID)	Global Division of instrument	`Division` (via `_globalDivision`)
`ibag[n]` (has sampleID)	Instrument zone (links to a sample)	`Division` (in `_divisions` list)
`phdr[i]`	Preset header	`InstPrst`
`pbag[0]` (no instrument gen)	Global Division of preset	`Division` (via `_globalDivision`)
`pbag[n]` (has instrument gen)	Preset zone (links to an instrument)	`Division` (in `_divisions` list)

Generators

Generators are simply `(AttributeType, AttributeValue)` pairs. There are 60 defined generator types (0 through `champ_endOper = 60`). Each is a tunable parameter such as pitch, filter cutoff, ADSR times, loop offsets, etc.

Source: `sources/core/types/attribute.h` lines 32–93 (the full `AttributeType` enum)

Modulators

A Modulator describes a MIDI-controlled mapping from a source signal to a destination generator:

```
Source → [shape function] → scale × amount → Destination generator
```

Components:

- `srcOper`: The primary MIDI source (`SFModulator`). Encodes: source index (CC number or general controller), curve shape (linear/concave/convex/switch), polarity (unipolar/bipolar), direction (ascending/descending)
- `amtSrcOper`: Optional secondary modulator that scales the `amount`
- `amount` (int16): Scaling factor applied to the mapped output
- `destOper` (uint16): Either an `AttributeType` (< 32768) or the index of another modulator (chained modulator)
- `transOper`: `linear` or `absolute_value`

Source: `sources/core/types/modulatordata.h`

---

3. SFZ Internal Structure

Overview of SFZ Parsing

SFZ is a plain-text format. Polyphone parses it in `InputParserSfz::processInternal()` and converts it entirely into the same internal SF2-like data model. After import, an SFZ is stored identically to an SF2 file internally.

Source: `sources/core/input/sfz/inputparsersfz.cpp`

SFZ Block Hierarchy

SFZ files contain four block types recognized by Polyphone:

```
<control>    – global file settings (default_path)
<global>     – opcodes that apply to ALL groups and regions
<group>      – opcodes that apply to all regions in this group
```

```
<region>     — one mapping: sample file + key range + vel range +
opcodes
```

These map to Polyphone's internal types during `changeBloc()`:

```
// inputparsersfz.cpp lines 226–248
if (bloc == "control")   _currentBloc = BLOC_CONTROL;
if (bloc == "global")    _currentBloc = BLOC_GLOBAL;
if (bloc == "group")  {  _currentBloc = BLOC_GROUP;
                          _presetList << SfzParameterGroup(); }
if (bloc == "region") {  _currentBloc = BLOC_REGION;
                          _presetList.last().newGroup(); }
```

SFZ Data Structures in Polyphone

SFZ Concept	Polyphone Class	File
`<global>` opcodes	`SfzParameterRegion _globalZone`	`inputparsersfz.h`
`<group>`	`SfzParameterGroup`	`sfzparametergroup.h`
`<region>` within a group	`SfzParameterRegion`	`sfzparameterregion.h`
An individual opcode	`SfzParameter`	`sfzparameter.h`

Inheritance Resolution

Polyphone resolves SFZ inheritance through a cascade of `mergeIfNotDefined` calls before decoding:

Step 1: Global zone opcodes are pushed into each group, only if the group has not already defined them:

```
// inputparsersfz.cpp lines 51
_presetList[i].moveOpcodesInGlobal(_globalZone);
```

Step 2: Group-level opcodes are pushed down into each region that hasn't defined them:

```
// sfzparametergroup.cpp: moveOpcodeInSamples()
if (_paramGlobaux.isDefined(opcode)) {
    value = _paramGlobaux.getDoubleValue(opcode);
    for (int i = 0; i < _regionList.size(); i++)
        if (!_regionList.at(i).isDefined(opcode))  // only if region
```

```
hasn't set it
            _regionList[i] << SfzParameter(opcode, value);
}
```

Rule: Region-level values always win. Group-level values apply only if the region doesn't define them. Global-level values apply only if neither group nor region defines them.

This is the SFZ inheritance cascade: `Global → Group → Region` (later levels override earlier ones).

Special Opcode Handling

Several opcodes must be attached to the region (sample zone), not the global division, because they depend on per-sample data:

```
// inputparsersfz.cpp lines 55-73
_presetList[i].moveOpcodeInSamples(SfzParameter::op_sample,
QMetaType::QString);
_presetList[i].moveOpcodeInSamples(SfzParameter::op_offset,
QMetaType::Int);
_presetList[i].moveOpcodeInSamples(SfzParameter::op_loop_start,
QMetaType::Int);
_presetList[i].moveOpcodeInSamples(SfzParameter::op_loop_end,
QMetaType::Int);
_presetList[i].moveOpcodeInSamples(SfzParameter::op_pan,
QMetaType::Double);
_presetList[i].moveOpcodeInSamples(SfzParameter::op_off_by,
QMetaType::Int);
```

SFZ → SF2 Mapping After Import

After inheritance resolution, Polyphone calls `SfzParameterGroup::decode()` which creates:

- One SF2 **Instrument** per SFZ `<group>`
- One SF2 **Sample** per unique audio file referenced
- One SF2 **Division (zone)** per SFZ `<region>`
- One SF2 **Preset** for the entire SFZ file (bank/preset derived from filename)

Source: `sources/core/input/sfz/sfzparametergroup.cpp` lines 222–355

SFZ OpCodes Recognized by Polyphone

`SfzParameter::OpCode` enum covers:

```
op_sample, op_key, op_keyMin, op_keyMax, op_velMin, op_velMax,
op_chanMin, op_chanMax, op_rootKey, op_exclusiveClass, op_off_by,
op_tuningFine, op_tuningCoarse, op_delay, op_offset, op_end,
op_loop_start, op_loop_end, op_loop_mode, op_trigger,
op_volume, op_pan, op_width, op_position, op_tuningScale,
op_chorus, op_reverb,
op_filterType, op_filterFreq, op_filterQ, op_fil_veltrack,
op_fil_keytrack, op_fil_keycenter,
op_amp_veltrack, op_amp_velcurve_1, op_amp_velcurve_127,
op_ampeg_delay, op_ampeg_attack, op_ampeg_hold, op_ampeg_decay,
op_ampeg_sustain, op_ampeg_release,
op_noteToVolEnvHold, op_noteToVolEnvDecay,
op_modLFOdelay, op_modLFOfreq, op_modLFOtoVolume,
op_pitcheg_delay, op_pitcheg_attack, op_pitcheg_hold, op_pitcheg_decay,
op_pitcheg_sustain, op_pitcheg_release,
op_noteToModEnvHold, op_noteToModEnvDecay, op_modEnvToTon,
op_vibLFOdelay, op_vibLFOfreq, op_vibLFOtoTon,
op_fileg_delay, op_fileg_attack, op_fileg_hold, op_fileg_decay,
op_fileg_sustain, op_fileg_release,
op_modEnvToFilter, op_fileg_holdcc133, op_fileg_decaycc133,
op_filLFOdelay, op_filLFOfreq, op_modLFOtoFilter
```

Source: `sources/core/input/sfz/sfzparameter.h`

---

4. Sample Representation

The `Smpl` Class

File: `sources/core/model/smpl.h`

```
class Smpl : public TreeItem {
    Sound         _sound;          // All audio data and properties
    quint16       _wSampleLink;    // Index of stereo partner (0 if
none)
    SFSampleLink  _sfSampleType;   // mono / left / right / linked / ROM
variants
    QString       _name;           // Display name (max 20 chars in SF2)
};
```

The `Sound` Class (holds actual data)

File: `sources/core/sample/sound.h`

`Sound` wraps the actual audio bytes and their metadata. It can store both 16-bit and 24-bit sample data, plus raw compressed data for SF3.

```
class Sound {
    QString  _fileName;     // Path to source file
    InfoSound _info;        // All descriptive metadata (see below)
    qint16*  _data16;       // 16-bit PCM samples (heap allocated)
    quint8*  _data24;       // Extra bytes for 24-bit samples
    char*    _rawData;      // Compressed (SF3/OGG) raw data
};
```

The `InfoSound` Class (sample metadata)

File: `sources/core/sample/infosound.h`

This is the complete set of metadata that describes a sample:

```
class InfoSound {
    quint32  dwStart;          // Start of 16-bit data in file (byte
offset)
    quint32  dwStart2;         // Start of 24-bit extension data in file
    quint32  dwLength;         // Number of samples (frames)
    quint32  dwSampleRate;     // Playback sample rate in Hz (e.g.,
44100)
    quint16  wChannels;        // 1 = mono, 2 = stereo
    quint16  wBpsFile;         // Bytes per sample value (2 = 16-bit, 3
= 24-bit)
    QList<QPair<quint32,quint32>> loops; // Loop points (list of
start/end pairs)
    quint32  dwRootKey;        // MIDI note number of original pitch
(default: 60 = Middle C)
    quint16  wChannel;         // Which channel this Smpl represents
(0=L, 1=R)
    qint16   iFineTune;        // Fine pitch correction (-100 to +100
cents)
    bool     pitchDefined;     // Whether root key was explicitly
detected
    bool     rawDataAvailable;// Whether raw (compressed) data can be
used
};
```

Sample Fields — Full Explanation

Field	`AttributeType` enum	Unit	Description
Name	`champ_name`	string	Human-readable label, max 20 chars in SF2
Root key	`champ_byOriginalPitch`	MIDI note (0–127)	The pitch at which the sample was recorded. Default: 60 (Middle C)
Fine tuning	`champ_chPitchCorrection`	cents (−99 to +99)	Correction applied on top of root key
Sample rate	`champ_dwSampleRate`	Hz	How many samples per second. Must never be 0
Length	`champ_dwLength`	samples	Total number of sample frames
Loop start	`champ_dwStartLoop`	sample offset	Stored relative to sample start
Loop end	`champ_dwEndLoop`	sample offset	Stored relative to sample start
16-bit data offset	`champ_dwStart16`	byte offset	Absolute position in file for 16-bit data
24-bit data offset	`champ_dwStart24`	byte offset	Absolute position of the extra-byte (sm24 chunk)
Bits per sample	`champ_bpsFile`	16 or 24	Bit depth of the stored data
Sample type	`champ_sfSampleType`	`SFSampleLink` enum	mono, left, right, linked, or ROM variants
Sample link	`champ_wSampleLink`	sample index	Points to stereo partner sample
Channel	`champ_wChannel`	0 or 1	Which channel of the source file this is

The `SFSampleLink` Enum

```
// attribute.h lines 152-162
typedef enum {
    linkInvalid     = 0,
```

```
    monoSample     = 1,
    rightSample    = 2,
    leftSample     = 4,
    linkedSample   = 8,
    RomMonoSample  = 0x8001,
    RomRightSample = 0x8002,
    RomLeftSample  = 0x8004,
    RomLinkedSample= 0x8008
} SFSampleLink;
```

Embedded Notes — Sample Level

> **ESP32 Relevance:** High.
>
> - `dwSampleRate`, `dwLength`, `dwStartLoop`, `dwEndLoop`, `byOriginalPitch`, `chPitchCorrection` are all required at runtime.
> - `wSampleLink` matters only if we want stereo playback. The ESP32 has limited RAM, so mono-only instruments are a realistic simplification for VHL.
> - `SFSampleLink` (left/right/mono) matters for pan at the sample level — needed if stereo.
> - ROM variants (`0x8001` ...) reference factory samples stored in synthesizer ROM; irrelevant for ESP32.
> - `dwStart16` / `dwStart24` / `bpsFile` are file-format storage details, not runtime parameters.
> - `rawDataAvailable` / `rawData` relate to SF3 compression — not needed for ESP32.

---

5. Region / Division Representation

The `Division` Class

File: `sources/core/model/division.h`

A `Division` is Polyphone's universal zone container. It is used for:

- An instrument zone (links to a sample, element type `elementInstSmpl`)
- A preset zone (links to an instrument, element type `elementPrstInst`)
- A global division (no link, just default parameters)

```
class Division : public TreeItem {
    // Operations on parameters
    bool isSet(AttributeType champ);
    void setGen(AttributeType champ, AttributeValue value);
    void resetGen(AttributeType champ);
    AttributeValue getGen(AttributeType champ);
    AttributeValue * getAttributeValues();  // array of [champ_endOper]
values
    bool           * getAttributeSet();     // which of those are
explicitly set

    // Operations on modulators
    int addMod();
    Modulator * getMod(int index);

    // Internal storage
    AttributeValue * _attributeValues;  // [champ_endOper] array, heap-
allocated
    bool           * _attributeSet;     // [champ_endOper] array, heap-
allocated
    IndexedElementList<Modulator *> _modulators;
    bool _mute;
};
```

How Parameters Are Stored in a Division

A `Division` allocates a fixed-size array of `AttributeValue` of length `champ_endOper` (= 60). A parallel boolean array `_attributeSet` tracks which parameters have been explicitly set. Unset parameters fall back to their default value.

This is a sparse representation: only a few of the 60 possible generators are typically set in any given zone.

Parameters in a Region (Instrument Zone)

The following parameters are zone-level because they control **how a specific sample is mapped and modified** for one note range:

`AttributeType`	Description	Unit	Default
`champ_keyRange`	Key range [lo, hi]	MIDI notes 0–127	[0, 127]
`champ_velRange`	Velocity range [lo, hi]	MIDI velocity 0–127	[0, 127]

AttributeType	Description	Unit	Default
champ_sampleID	Index of the sample	sample index	— (must be set)
champ_overridingRootKey	Override root key for pitch calculation	MIDI note (−1 = use sample's root key)	−1
champ_coarseTune	Coarse pitch offset	semitones (−120 to +120)	0
champ_fineTune	Fine pitch offset (added to sample fine tune)	cents (−99 to +99)	0
champ_scaleTuning	How pitch changes per MIDI key	cents/key (0–1200, default 100 = equal temperament)	100
champ_initialAttenuation	Volume reduction	dB × 0.4 (see note)	0
champ_pan	Stereo position	−500 (full left) to +500 (full right), 0 = center	0
champ_initialFilterFc	Low-pass filter cutoff	Hz (1500–13500 Hz for instruments)	13500
champ_initialFilterQ	Filter resonance	dB (0–96 dB)	0
champ_sampleModes	Loop mode	0=no loop, 1=loop, 2=no loop until note off, 3=loop until note off	0
champ_exclusiveClass	Exclusive group (mutes others in same class)	0–127 (0 = no class)	0
champ_startAddrsOffset	Sample start offset (fine)	samples (−32767 to	0

AttributeType	Description	Unit	Default
		+32767)	
champ_startAddrsCoarseOffset	Sample start offset (coarse)	32768-sample units	0
champ_endAddrsOffset	Sample end offset (fine)	samples	0
champ_endAddrsCoarseOffset	Sample end offset (coarse)	32768-sample units	0
champ_startloopAddrsOffset	Loop start offset (fine)	samples	0
champ_startloopAddrsCoarseOffset	Loop start offset (coarse)	32768-sample units	0
champ_endloopAddrsOffset	Loop end offset (fine)	samples	0
champ_endloopAddrsCoarseOffset	Loop end offset (coarse)	32768-sample units	0
champ_keynum	Force fixed playback key	−1 = use played key; 0–127 = always play this key	−1
champ_velocity	Force fixed velocity	−1 = use actual velocity; 0–127 = fixed	−1

Volume Envelope — Defined in a Region

AttributeType	Description	Unit	Default
champ_delayVolEnv	Time before attack begins	seconds (timecent internally)	0.001 s
champ_attackVolEnv	Time for volume to rise from 0 dB to full	seconds	0.001 s
champ_holdVolEnv	Time volume stays at full after attack	seconds	0.001 s

AttributeType	Description	Unit	Default
champ_decayVolEnv	Time for volume to fall from full to sustain	seconds	0.001 s
champ_sustainVolEnv	Sustain level (attenuation from full)	dB	0 (no attenuation = full sustain)
champ_releaseVolEnv	Time for volume to fall to silence after key off	seconds	0.001 s
champ_keynumToVolEnvHold	How much the held key number modifies hold time	cents/key	0
champ_keynumToVolEnvDecay	How much the held key number modifies decay time	cents/key	0

Modulation Envelope — Defined in a Region

AttributeType	Description	Unit	Default
champ_delayModEnv	Delay before mod env attack	seconds	0.001 s
champ_attackModEnv	Mod env attack time	seconds	0.001 s
champ_holdModEnv	Mod env hold time	seconds	0.001 s
champ_decayModEnv	Mod env decay time	seconds	0.001 s
champ_sustainModEnv	Mod env sustain level	% (0–100%)	0
champ_releaseModEnv	Mod env release time	seconds	0.001 s
champ_keynumToModEnvHold	Key number effect on mod env hold	cents/key	0
champ_keynumToModEnvDecay	Key number effect on mod env decay	cents/key	0
champ_modEnvToPitch	How much mod env affects pitch	cents	0
champ_modEnvToFilterFc	How much mod env affects filter cutoff	cents	0

LFOs — Defined in a Region

AttributeType	Description	Unit	Default
champ_delayModLFO	Mod LFO start delay	seconds	0.001 s
champ_freqModLFO	Mod LFO frequency	Hz	8.176 Hz
champ_modLfoToPitch	Mod LFO → pitch depth	cents	0
champ_modLfoToVolume	Mod LFO → volume depth	dB	0
champ_modLfoToFilterFc	Mod LFO → filter cutoff depth	cents	0
champ_delayVibLFO	Vib LFO start delay	seconds	0.001 s
champ_freqVibLFO	Vib LFO frequency	Hz	8.176 Hz
champ_vibLfoToPitch	Vib LFO → pitch depth	cents	0

Effects — Defined in a Region

AttributeType	Description	Unit	Default
champ_chorusEffectsSend	Chorus send amount	% (0–100)	0
champ_reverbEffectsSend	Reverb send amount	% (0–100)	0

Why These Belong to the Region, Not the Sample

A single **sample** can be used by multiple **regions** (zones) with completely different envelope shapes, filter settings, and loop modes. The region is the customization layer that lets the same raw audio function differently across different key ranges, velocity layers, or even different instruments entirely.

Embedded Notes — Region/Division Level

> **ESP32 Relevance:** Critical — all parameters in this section are required at runtime.
>
> - Key range, velocity range, sampleID, root key: required for note-on zone selection.
> - Envelope DAHDSR × 2 (volume + modulation): required for synthesis.
> - Filter cutoff and Q: required if filtering is implemented.
> - Pan: required for stereo output; can simplify to mono only.
> - Tuning (coarse, fine, scale): required for pitch computation.
> - Loop mode + sample offsets: required for looped playback.
> - LFOs: desirable but can potentially be simplified or deferred to Module 2.

- Chorus/reverb sends: lower priority for ESP32; likely deferred.

---

6. Group / Global Division Representation

What a Global Division Is

Every `InstPrst` object (both Instruments and Presets) has exactly **one Global Division** in addition to its list of zone divisions.

```
// instprst.h line 44
Division * getGlobalDivision() { return _globalDivision; }
```

The Global Division is a normal `Division` object. Its parameters serve as **default values** for all zones within the instrument. Any zone that does not set a parameter inherits it from the Global Division.

Inheritance Implementation

In `VoiceParam::readDivisionAttributes()`, the reading is done in two passes: first global, then zone. The zone pass can override any parameter:

```
// voiceparam.cpp lines 197-220
void VoiceParam::readDivisionAttributes(Division * globalDivision,
Division * division, bool isPrst)
{
    // Pass 1: configure with the global attributes
    bool * attributeSet = globalDivision->getAttributeSet();
    AttributeValue * attributeValues = globalDivision-
>getAttributeValues();
    for (int i = 0; i < champ_endOper; i++) {
        if (attributeSet[i])
            _parameters[i].initValue(attributeValues[i], isPrst);
    }

    // Pass 2: configure with the division attributes (overriding
global)
    attributeSet   = division->getAttributeSet();
    attributeValues = division->getAttributeValues();
    for (int i = 0; i < champ_endOper; i++) {
        if (attributeSet[i])
            _parameters[i].initValue(attributeValues[i], isPrst);
```

```
    }
}
```

Preset vs Instrument Layers

There are two levels of global/zone hierarchy:

1. **Instrument level** (first): reads sample properties, then instrument global division, then instrument zone.
2. **Preset level** (second, additive): reads preset global division, then preset zone, and these **add to** (not replace) the instrument-level values.

```
// voiceparam.cpp lines 74-82
Division * instGlobalDiv = (inst != nullptr ? inst->getGlobalDivision()
: nullptr);
if (instDiv)
    readDivisionAttributes(instGlobalDiv, instDiv, false);    // inst
level

Division * prstGlobalDiv = (prst != nullptr ? prst->getGlobalDivision()
: nullptr);
if (prstDiv)
    readDivisionAttributes(prstGlobalDiv, prstDiv, true);    // prst
level (additive)
```

The `isPrst` flag changes whether a parameter is treated as an absolute value (instrument) or an additive offset (preset). This is the key distinction between instrument and preset layers.

What Groups Can Contain (in Polyphone's Terms)

Both the Global Division and zone Divisions can contain:

- Any generator parameter (`champ_*` values 0–59)
- Any number of modulators (`Modulator` objects)

There is no distinct "group" class. The term "global division" is Polyphone's equivalent.

Modulator Inheritance

Modulators are loaded from both the global division and the zone division:

```
// voiceparam.cpp lines 222-252
// Load global modulators
```

```
QVector<Modulator *> modulators = globalDivision->getMods().values();
foreach (Modulator * mod, modulators)
    if (!mod->isHidden())
        _modulatorGroupInst.loadModulators(&mod->_data, 1);

// Load division modulators (in addition to global modulators)
modulators = division->getMods().values();
foreach (Modulator * mod, modulators)
    if (!mod->isHidden())
        _modulatorGroupInst.loadModulators(&mod->_data, 1);
```

Modulators are **additive**: zone modulators do not replace global modulators; both sets are active.

Embedded Notes — Group / Global Division Level

> **ESP32 Relevance:** Medium.
>
> - The concept of global division as default-value storage is important for the VHL format.
> - At runtime, we can pre-flatten (bake) global + zone into a single set of zone parameters. This saves a pointer dereference per note-on.
> - Preset-level additive offsets require careful design: we need to decide whether to bake them or apply them dynamically at note-on time.

---

7. Instrument Parameters — Complete Table

This is derived from `VoiceParam::prepareParameters()` in `voiceparam.cpp`, which documents every parameter that actually gets used at runtime.

Parameter Groups

A. Sample Address Offsets (instrument only)

Parameter	`AttributeType`	Unit	Purpose
Sample start fine offset	`champ_startAddrsOffset`	samples	Offset added to start of sample
Sample start coarse offset	`champ_startAddrsCoarseOffset`	×32768 samples	Large start offset

Parameter	AttributeType	Unit	Purpose
Sample end fine offset	champ_endAddrsOffset	samples	Offset subtracted from end
Sample end coarse offset	champ_endAddrsCoarseOffset	×32768 samples	Large end offset
Loop start fine offset	champ_startloopAddrsOffset	samples	Loop start adjustment
Loop start coarse offset	champ_startloopAddrsCoarseOffset	×32768 samples	Large loop start adjustment
Loop end fine offset	champ_endloopAddrsOffset	samples	Loop end adjustment
Loop end coarse offset	champ_endloopAddrsCoarseOffset	×32768 samples	Large loop end adjustment

B. Volume Envelope

Parameter	AttributeType	Unit	Default (inst)
Vol env delay	champ_delayVolEnv	seconds	0.001 s
Vol env attack	champ_attackVolEnv	seconds	0.001 s
Vol env hold	champ_holdVolEnv	seconds	0.001 s
Vol env decay	champ_decayVolEnv	seconds	0.001 s
Vol env sustain	champ_sustainVolEnv	dB (0 = full sustain)	0
Vol env release	champ_releaseVolEnv	seconds	0.001 s
Key → vol env hold	champ_keynumToVolEnvHold	cents/key	0
Key → vol env decay	champ_keynumToVolEnvDecay	cents/key	0

C. Modulation Envelope

Parameter	AttributeType	Unit	Default (inst)
Mod env delay	champ_delayModEnv	seconds	0.001 s

Parameter	AttributeType	Unit	Default (inst)
Mod env attack	champ_attackModEnv	seconds	0.001 s
Mod env hold	champ_holdModEnv	seconds	0.001 s
Mod env decay	champ_decayModEnv	seconds	0.001 s
Mod env sustain	champ_sustainModEnv	%	0
Mod env release	champ_releaseModEnv	seconds	0.001 s
Key → mod env hold	champ_keynumToModEnvHold	cents/key	0
Key → mod env decay	champ_keynumToModEnvDecay	cents/key	0
Mod env → pitch	champ_modEnvToPitch	cents	0
Mod env → filter	champ_modEnvToFilterFc	cents	0

D. Modulation LFO

Parameter	AttributeType	Unit	Default (inst)
Mod LFO delay	champ_delayModLFO	seconds	0.001 s
Mod LFO frequency	champ_freqModLFO	Hz	8.176 Hz
Mod LFO → pitch	champ_modLfoToPitch	cents	0
Mod LFO → filter	champ_modLfoToFilterFc	cents	0
Mod LFO → volume	champ_modLfoToVolume	dB	0

E. Vibrato LFO

Parameter	AttributeType	Unit	Default (inst)
Vib LFO delay	champ_delayVibLFO	seconds	0.001 s
Vib LFO frequency	champ_freqVibLFO	Hz	8.176 Hz
Vib LFO → pitch	champ_vibLfoToPitch	cents	0

F. Filter

Parameter	AttributeType	Unit	Default (inst)
Filter cutoff	champ_initialFilterFc	Hz (1500–13500 for inst)	13500 Hz (fully open)

Parameter	AttributeType	Unit	Default (inst)
Filter resonance	champ_initialFilterQ	dB (0–96)	0

G. Effects and Pan

Parameter	AttributeType	Unit	Default
Chorus send	champ_chorusEffectsSend	% (0–100)	0
Reverb send	champ_reverbEffectsSend	% (0–100)	0
Pan	champ_pan	−500 (left) to +500 (right)	0 (center)

H. Tuning

Parameter	AttributeType	Unit	Default
Coarse tune	champ_coarseTune	semitones (−120 to +120)	0
Fine tune	champ_fineTune	cents (−99 to +99)	0
Scale tuning	champ_scaleTuning	cents/key	100 (equal temperament)

I. Playback Control

Parameter	AttributeType	Unit / Values	Default
Overriding root key	champ_overridingRootKey	MIDI note (−1 = use sample root)	−1
Fixed key	champ_keynum	MIDI note (−1 = use played key)	−1
Fixed velocity	champ_velocity	0–127 (−1 = use actual vel)	−1
Loop mode	champ_sampleModes	0=no loop, 1=continuous, 2=until note off, 3=loop until release	0
Exclusive class	champ_exclusiveClass	0–127 (0 = no class)	0

Note on `initialAttenuation` Unit Conversion

Polyphone applies a historical scaling factor `DB_SF2_TO_REAL_DB = 0.4` to `initialAttenuation`:

```
// attribute.cpp line 149-150
case champ_initialAttenuation:
    realValue = 0.1 * DB_SF2_TO_REAL_DB * static_cast<double>
(storedValue.shValue);
```

So stored value 100 = 0.1 × 0.4 × 100 = 4 dB. This is a known deviation from the SF2 spec (which uses 0.1 dB per unit). **When designing VHL, use real dB directly and avoid this historical error.**

---

8. Runtime vs Editor Parameters

Editor-Only Parameters (do not affect sound)

Parameter	`AttributeType`	Where Stored	Reason
Bank name	`champ_name` on `elementSf2`	`Soundfont._INAM`	Metada
SF2 version	`champ_IFIL`	`Soundfont._IFIL`	File forn info
Sound engine	`champ_ISNG`	`Soundfont._ISNG`	Target s identific
ROM name	`champ_IROM`	`Soundfont._IROM`	Metada
ROM version	`champ_IVER`	`Soundfont._IVER`	Metada
Creation date	`champ_ICRD`	`Soundfont._ICRD`	Metada
Engineer	`champ_IENG`	`Soundfont._IENG`	Metada
Product	`champ_IPRD`	`Soundfont._IPRD`	Metada
Copyright	`champ_ICOP`	`Soundfont._ICOP`	Metada
Comments	`champ_ICMT`	`Soundfont._ICMT`	Editor commer
Tools used	`champ_ISFT`	`Soundfont._ISFT`	Metada
File path	`champ_filenameInitial`	`Soundfont._fileNameInitial`	File manage

Parameter	`AttributeType`	Where Stored	Reason
Data file	`champ_filenameForData`	`Soundfont._fileNameForData`	File I/O
Bits/sample (file)	`champ_wBpsInit`	`Soundfont._wBpsInit`	Save fo
Bits/sample (save)	`champ_wBpsSave`	`Soundfont._wBpsSave`	Save fo
Preset bank number	`champ_wBank`	`InstPrst._extraFields`	MIDI ro
Preset program number	`champ_wPreset`	`InstPrst._extraFields`	MIDI ro
Library	`champ_dwLibrary`	`InstPrst._extraFields`	SF2 fiel (unused practice
Genre	`champ_dwGenre`	`InstPrst._extraFields`	SF2 fiel (unused practice
Morphology	`champ_dwMorphology`	`InstPrst._extraFields`	SF2 fiel (unused practice
Sample name	`champ_name` on `elementSmpl`	`Smpl._name`	UI displ
Instrument name	`champ_name` on `elementInst`	`InstPrst._name`	UI displ
Preset name	`champ_name` on `elementPrst`	`InstPrst._name`	UI displ
Mute flag	`champ_mute`	`Division._mute`	Editor te
Always play flag	`champ_alwaysPlayed`	`InstPrst._alwaysPlay`	Editor te
Modulator hidden flag	`Modulator._hidden`	`Modulator._hidden`	Editor st
Raw data available	`champ_rawDataAvailable`	`InfoSound.rawDataAvailable`	SF3 forr flag
Pitch defined	`champ_pitchDefined`	`InfoSound.pitchDefined`	Auto-de flag

Runtime-Required Parameters (affect sound)

Parameter	`AttributeType`	Category
Root key	`champ_byOriginalPitch`	Sample
Fine pitch correction	`champ_chPitchCorrection`	Sample
Sample rate	`champ_dwSampleRate`	Sample
Sample length	`champ_dwLength`	Sample
Loop start	`champ_dwStartLoop`	Sample
Loop end	`champ_dwEndLoop`	Sample
Sample type	`champ_sfSampleType`	Sample
Sample link	`champ_wSampleLink`	Sample
Sample data (16-bit)	`champ_sampleData`	Sample
Sample data (24-bit ext)	`champ_sampleRawData`	Sample
Key range	`champ_keyRange`	Zone routing
Velocity range	`champ_velRange`	Zone routing
Sample ID	`champ_sampleID`	Zone routing
Instrument ID	`champ_instrument`	Zone routing
Overriding root key	`champ_overridingRootKey`	Tuning
Coarse tune	`champ_coarseTune`	Tuning
Fine tune	`champ_fineTune`	Tuning
Scale tuning	`champ_scaleTuning`	Tuning
Initial attenuation	`champ_initialAttenuation`	Volume
Pan	`champ_pan`	Stereo
Filter cutoff	`champ_initialFilterFc`	Filter
Filter resonance	`champ_initialFilterQ`	Filter
All vol env stages	`champ_delay/attack/hold/decay/sustainVolEnv` , `champ_releaseVolEnv`	Envelope

Parameter	`AttributeType`	Category
Key → vol env hold/decay	`champ_keynumToVolEnvHold/Decay`	Envelope
All mod env stages	`champ_delay/attack/hold/decay/sustainModEnv`, `champ_releaseModEnv`	Envelope
Key → mod env hold/decay	`champ_keynumToModEnvHold/Decay`	Envelope
Mod env → pitch/filter	`champ_modEnvToPitch/FilterFc`	Envelope routing
All mod LFO params	`champ_delayModLFO`, `champ_freqModLFO`, LFO-to-X	LFO
All vib LFO params	`champ_delayVibLFO`, `champ_freqVibLFO`, `champ_vibLfoToPitch`	LFO
Loop mode	`champ_sampleModes`	Playback
Exclusive class	`champ_exclusiveClass`	Polyphony
Fixed key	`champ_keynum`	Override
Fixed velocity	`champ_velocity`	Override
Sample address offsets	all 8 `Addrs*Offset` fields	Playback
Chorus/reverb send	`champ_chorusEffectsSend`, `champ_reverbEffectsSend`	FX
All modulators	`ModulatorData` objects	Dynamic routing

---

9. Data Structures — Class Reference

`Soundfonts`

File: `sources/core/model/soundfonts.h`

Root container. Holds a list of all open `Soundfont` objects. Accessed through `SoundfontManager::getSoundfonts()`.

`Soundfont`

File: `sources/core/model/soundfont.h`

One open soundfont file. Contains:

- `SfVersionTag _IFIL` — SF2 version (major.minor), e.g., 2.4
- `QString _ISNG, _INAM, _IROM, _ICRD, _IENG, _IPRD, _ICOP, _ICMT, _ISFT` — metadata strings
- `QString _fileNameInitial, _fileNameForData` — file paths
- `quint16 _wBpsInit, _wBpsSave` — bit depth (16 or 24)
- `IndexedElementList<Smpl *> _smpl` — all samples
- `IndexedElementList<InstPrst *> _inst` — all instruments
- `IndexedElementList<InstPrst *> _prst` — all presets

`Smpl`

File: `sources/core/model/smpl.h`

Represents one sample entry. Contains:

- `Sound _sound` — audio data and metadata (see `InfoSound`)
- `quint16 _wSampleLink` — index of stereo partner
- `SFSampleLink _sfSampleType` — mono/left/right/linked/ROM
- `QString _name` — display name

`Sound`

File: `sources/core/sample/sound.h`

Audio data container. Owns the actual PCM bytes and reads them lazily from file:

- `InfoSound _info` — all descriptive metadata
- `qint16 * _data16` — 16-bit PCM, or `nullptr` if not loaded
- `quint8 * _data24` — extra byte for 24-bit, or `nullptr`
- `char * _rawData` — compressed (SF3) raw bytes
- `SampleReader * _reader` — the reader used to decode this format

Provides `getDataFloat()` and `getData()` for DSP access.

`InfoSound`

File: `sources/core/sample/infosound.h`

Plain struct, no inheritance. Fields documented in Section 4.

`InstPrst`

File: `sources/core/model/instprst.h`

Shared class for both Instruments and Presets. The distinction is entirely in the `ElementType` of the `EltID`.

Members:

- `Division * _globalDivision` — the global zone (default parameters)
- `IndexedElementList<Division *> _divisions` — list of sample/instrument zones
- `QString _name` — display name
- `int _extraFields[5]` — preset-only fields: `champ_wPreset`, `champ_wBank`, `champ_dwLibrary`, `champ_dwGenre`, `champ_dwMorphology`
- `bool _alwaysPlay` — editor testing flag (not runtime)

`Division`

File: `sources/core/model/division.h`

One zone. Can be a global division, an instrument zone (links to a sample), or a preset zone (links to an instrument).

Members:

- `AttributeValue * _attributeValues` — heap array of `champ_endOper` (60) values
- `bool * _attributeSet` — parallel heap array of 60 bools, tracks which are set
- `IndexedElementList<Modulator *> _modulators` — list of modulators attached to this zone
- `bool _mute` — editor-only: mute this zone for testing

`Modulator`

File: `sources/core/model/modulator.h`

```
class Modulator {
    ModulatorData _data;   // The actual modulator specification
    int _id;               // Internal index
    bool _hidden;          // Editor-only: hidden from view
};
```

`ModulatorData`

File: `sources/core/types/modulatordata.h`

```
class ModulatorData {
    SFModulator srcOper;      // Primary source (MIDI CC, velocity,
pressure, etc.)
    quint16     destOper;     // Destination generator (AttributeType
or chained mod index)
    SFModulator amtSrcOper;   // Secondary (amount scaling) source
    SFTransform transOper;    // linear or absolute_value
    qint16      amount;       // Scaling factor
    quint16     index;        // Global modulator index (for chained
references)
};
```

SFModulator

File: `sources/core/types/modulatordata.h`

```
class SFModulator {
    quint8 Type      : 6; // Curve: linear, concave, convex, switch
    bool isBipolar   : 1; // unipolar (0→1) or bipolar (-1→1)
    bool isDescending: 1; // ascending (0→127=0→1) or descending
    bool CC          : 1; // true = MIDI CC source, false = general
controller
    quint8 Index     : 7; // CC number (0-127) or GeneralController
enum value
};
```

Attribute

File: `sources/core/types/attribute.h`

A wrapper that holds one parameter value with its `AttributeType` and stores both a "stored value" (raw binary from file) and a "real value" (human-readable units).

```
class Attribute {
    AttributeType  _champ;       // Which parameter this is
    AttributeValue _storedValue; // Raw binary representation
    double         _realValue;   // Human-readable units (Hz, dB,
seconds, etc.)
    bool           _isPrst;      // Whether this is in preset context
(affects scaling)
};
```

AttributeValue

File: `sources/core/types/attribute.h`

A union that can hold any of the SF2 primitive types:

```
union AttributeValue {
    quint8      bValue;     // unsigned 8-bit
    quint16     wValue;     // unsigned 16-bit
    quint32     dwValue;    // unsigned 32-bit
    qint8       cValue;     // signed 8-bit
    qint16      shValue;    // signed 16-bit
    RangesType  rValue;     // {byLo, byHi} for key/vel ranges
    SFModulator sfModValue; // for modulator source fields
    SfVersionTag sfVerValue;// for SF2 version tags
    SFSampleLink sfLinkValue;// for sample type
    SFTransform sfTransValue;// for modulator transform
};
```

`EltID`

File: `sources/core/types/eltid.h`

The universal identifier for any element in any soundfont:

```
class EltID {
    ElementType typeElement; // What kind of element: Sf2, Smpl, Inst,
Prst, InstSmpl, etc.
    int indexSf2;            // Which soundfont file
    int indexElt;            // Which element within that file
(sample/inst/preset index)
    int indexElt2;           // Which sub-element (zone index within an
instrument)
    int indexMod;            // Which modulator within a zone
};
```

The `ElementType` enum maps the full hierarchy:

```
elementSf2 = 0         // A whole soundfont
elementSmpl = 1        // A sample
elementInst = 2        // An instrument
elementPrst = 3        // A preset
elementInstSmpl = 4    // A zone within an instrument (links to sample)
elementPrstInst = 5    // A zone within a preset (links to instrument)
elementRootSmpl = 6    // The "Samples" folder node (tree UI)
elementRootInst = 7    // The "Instruments" folder node
elementRootPrst = 8    // The "Presets" folder node
elementInstMod = 9     // A modulator in an instrument global zone
elementPrstMod = 10    // A modulator in a preset global zone
```

```
elementInstSmplMod = 11 // A modulator in an instrument zone
elementPrstInstMod = 12 // A modulator in a preset zone
elementInstGen = 13   // (unused: generator query type)
...
```

VoiceParam

File: `sources/sound_engine/voiceparam.h`

The runtime view of all parameters for one voice (note). Populated at note-on from the instrument/sample/preset data.

```
class VoiceParam {
    ModulatedParameter _parameters[champ_endOper]; // All 60
parameters, with modulation applied
    ModulatorGroup _modulatorGroupInst;             // Instrument-level
modulator processing
    ModulatorGroup _modulatorGroupPrst;             // Preset-level
modulator processing
    qint32 _sampleLength, _sampleLoopStart, _sampleLoopEnd,
_sampleFineTune;

    // Identification
    qint8 _channel, _key, _type;
    int _sf2Id, _presetId;
    qint32 _wPresetNumber;
};
```

`VoiceParam::initialize()` is called at note-on time and performs:

1. Read sample properties (`readSmpl`)
2. Read instrument global + zone attributes (`readDivisionAttributes`)
3. Read preset global + zone attributes (`readDivisionAttributes` with `isPrst=true`)
4. Load modulators from both levels (`readDivisionModulators`)
5. Compute all initial modulations (`computeModulations`)

SoundfontManager

File: `sources/core/soundfontmanager.h`

Global singleton. The single access point for all read/write operations on soundfont data. All editors, parsers, and the sound engine communicate through this object.

```
class SoundfontManager : public QObject {
    static SoundfontManager * getInstance();

    // Universal get/set API
    AttributeValue get(EltID id, AttributeType champ);
    int set(EltID id, AttributeType champ, AttributeValue value);
    bool isSet(EltID id, AttributeType champ);
    void reset(EltID id, AttributeType champ);
    QString getQstr(EltID id, AttributeType champ);
    Sound * getSound(EltID id);

    // Add/delete elements
    int add(EltID id);
    void remove(EltID id, int *message = nullptr);

    // Access to raw data for the sound engine
    Soundfonts * getSoundfonts();
    QRecursiveMutex * getMutex();
};
```

---

10. Source Code Extraction Reference

Entry 1 — AttributeType Enum (complete generator list)

Source File	`sources/core/types/attribute.h`
Lines	30–149
Purpose	Defines all 60 SF2 generator codes (0–59) plus extended codes for sample data, preset metadata, modulator fields, and editor state
Why it matters	This enum is the Rosetta Stone for all parameter access throughout Polyphone. Every generator, every metadata field, and every editor flag is encoded here.

```
// Key subset (generators 0–59):
champ_startAddrsOffset = 0,
champ_endAddrsOffset = 1,
champ_startloopAddrsOffset = 2,
champ_endloopAddrsOffset = 3,
champ_startAddrsCoarseOffset = 4,
champ_modLfoToPitch = 5,
champ_vibLfoToPitch = 6,
```

```
champ_modEnvToPitch = 7,
champ_initialFilterFc = 8,
champ_initialFilterQ = 9,
champ_modLfoToFilterFc = 10,
champ_modEnvToFilterFc = 11,
champ_endAddrsCoarseOffset = 12,
champ_modLfoToVolume = 13,
champ_chorusEffectsSend = 15,
champ_reverbEffectsSend = 16,
champ_pan = 17,
champ_delayModLFO = 21, champ_freqModLFO = 22,
champ_delayVibLFO = 23, champ_freqVibLFO = 24,
champ_delayModEnv = 25, champ_attackModEnv = 26,
champ_holdModEnv = 27,  champ_decayModEnv = 28,
champ_sustainModEnv = 29, champ_releaseModEnv = 30,
champ_keynumToModEnvHold = 31, champ_keynumToModEnvDecay = 32,
champ_delayVolEnv = 33, champ_attackVolEnv = 34,
champ_holdVolEnv = 35,  champ_decayVolEnv = 36,
champ_sustainVolEnv = 37, champ_releaseVolEnv = 38,
champ_keynumToVolEnvHold = 39, champ_keynumToVolEnvDecay = 40,
champ_instrument = 41,    // Links preset zone to instrument
champ_keyRange = 43,
champ_velRange = 44,
champ_startloopAddrsCoarseOffset = 45,
champ_keynum = 46,        // Fixed key override
champ_velocity = 47,      // Fixed velocity override
champ_initialAttenuation = 48,
champ_endloopAddrsCoarseOffset = 50,
champ_coarseTune = 51,
champ_fineTune = 52,
champ_sampleID = 53,      // Links instrument zone to sample
champ_sampleModes = 54,   // Loop mode
champ_scaleTuning = 56,
champ_exclusiveClass = 57,
champ_overridingRootKey = 58,
champ_endOper = 60        // Sentinel: total count of generator types
```

Entry 2 — Default Stored Values for Generators

Source File	`sources/core/types/attribute.cpp`
Function	`Attribute::getDefaultStoredValue()`

Source File	`sources/core/types/attribute.cpp`
Lines	264–310
Purpose	Defines the default raw binary value for every generator when not explicitly set
Why it matters	These defaults are what a voice plays with if a parameter is not set in any division. Essential for the ESP32 fallback behavior.

```
// Key defaults:
champ_velRange / champ_keyRange:  byLo=0, byHi=127  (full range)
champ_keynum / champ_velocity / champ_overridingRootKey: shValue = -1
(not set)
champ_scaleTuning: shValue = isPrst ? 0 : 100  // inst=100 cents/key,
prst=0 (no change)
champ_initialFilterFc: wValue = isPrst ? 0 : 13500  // inst=fully open
// All envelopes: shValue = -12000 for inst (≈ 0.001 s), 0 for prst
// All others: 0
```

Entry 3 — Unit Conversion (Stored → Real)

Source File	`sources/core/types/attribute.cpp`
Function	`Attribute::toRealValue()`
Lines	116–179
Purpose	Converts binary stored values to human-readable real units (Hz, dB, seconds, etc.)
Why it matters	Defines the mathematical relationship between what is stored in the file and what the sound engine receives

```
// Timecent → seconds conversion (used for all envelope and LFO times):
realValue = pow(2, storedValue.shValue / 1200.0);
// So -12000 → pow(2, -10) ≈ 0.001 seconds
//      0    → pow(2,   0) = 1.000 second
//    12000  → pow(2, +10) = 1024 seconds

// Cents → Hz (for filter cutoff, LFO frequency):
realValue = pow(2, storedValue.shValue / 1200.0) * 8.176;
// 13500 stored → pow(2, 11.25) * 8.176 ≈ 20 kHz
```

```
// For pan, Q, sustain, LFO-to-volume:
realValue = 0.1 * storedValue.shValue;
// 500 → 50.0 (pan unit); 1000 → 100.0 (%)

// For initialAttenuation (with historical factor):
realValue = 0.1 * 0.4 * storedValue.shValue; // Historical error in SF2 spec
```

Entry 4 — Zone Discovery and Global/Zone Distinction

Source File	`sources/core/input/sf/inputparsersf.cpp`
Function	`InputParserSf::fillSf2()`
Lines	274–296
Purpose	Determines whether a bag is a global zone (no `champ_sampleID`) or a sample zone (has `champ_sampleID`)
Why it matters	This is the fundamental algorithm for reading SF2 bag structure

```
global = 1;
for (int k = genmin; k < genmax; k++) {
    if (pdtaPart._igens[k]._sfGenOper.value == champ_sampleID)
        global = 0;    // Zone has a sample → it's NOT global
}
// If global == 1: parameters go to instrument global division
// If global == 0: create a new zone, link to the sample
```

Entry 5 — Parameter Initialization at Note-On

Source File	`sources/sound_engine/voiceparam.cpp`
Function	`VoiceParam::initialize()`
Lines	45–111
Purpose	The complete note-on parameter collection sequence
Why it matters	Defines the exact order and precedence of all parameter sources

```
Order of precedence (later overrides earlier):
1. Defaults (from Attribute::getDefaultStoredValue)
2. Sample root key and fine tune (from Smpl._sound)
3. Instrument global division attributes
4. Instrument zone attributes (override global)
5. Preset global division attributes (additive on top)
6. Preset zone attributes (additive on top, override preset global)
7. Default modulators (SF2 spec mandated, 10 rules)
8. Instrument-level modulator results
9. Preset-level modulator results
```

Entry 6 — Default Modulators (SF2 Mandatory Rules)

Source File	`sources/core/types/modulatordata.cpp`
Function	`ModulatorData::loadDefaultModulator()`
Lines	185–270
Purpose	Defines the 10 mandatory SF2 default modulators that apply to every voice
Why it matters	These are always active and cannot be disabled unless explicitly overridden by a user modulator

The 10 mandatory SF2 default modulators:

#	Source	Amount	Destination
1	Velocity (concave, descending)	960	Initial Attenuation
2	Velocity (linear, descending)	−2400	Filter Cutoff
3	Channel Pressure (linear)	50	Vib LFO Pitch
4	CC 1 / Mod Wheel (linear)	50	Vib LFO Pitch
5	CC 7 / Volume (concave, descending)	960	Initial Attenuation
6	CC 10 / Pan (linear, bipolar)	1000	Pan
7	CC 11 / Expression (concave, descending)	960	Initial Attenuation
8	CC 91 / Reverb (linear)	200	Reverb Send

#	Source	Amount	Destination
9	CC 93 / Chorus (linear)	200	Chorus Send
10	Pitch Wheel (linear, bipolar) × PWS sensitivity	12700	Fine Tune

---

Entry 7 — SFZ Inheritance Push-Down

Source File	`sources/core/input/sfz/sfzparametergroup.cpp`
Function	`SfzParameterGroup::moveOpcodeInSamples()`
Lines	53–88
Purpose	Implements the "group → region" inheritance: copies a group-level opcode to any region that hasn't defined it
Why it matters	This is the actual SFZ inheritance algorithm

```
void SfzParameterGroup::moveOpcodeInSamples(SfzParameter::OpCode
opcode, QMetaType::Type type)
{
    if (_paramGlobaux.isDefined(opcode)) {
        // For each region that hasn't set this opcode: copy the group
value
        for (int i = 0; i < _regionList.size(); i++)
            if (!_regionList.at(i).isDefined(opcode))
                _regionList[i] << SfzParameter(opcode, value);
    }
    _paramGlobaux.removeOpCode(opcode); // Remove from group after
pushing down
}
```

Entry 8 — `getPosition()` : Final Runtime Position Computation

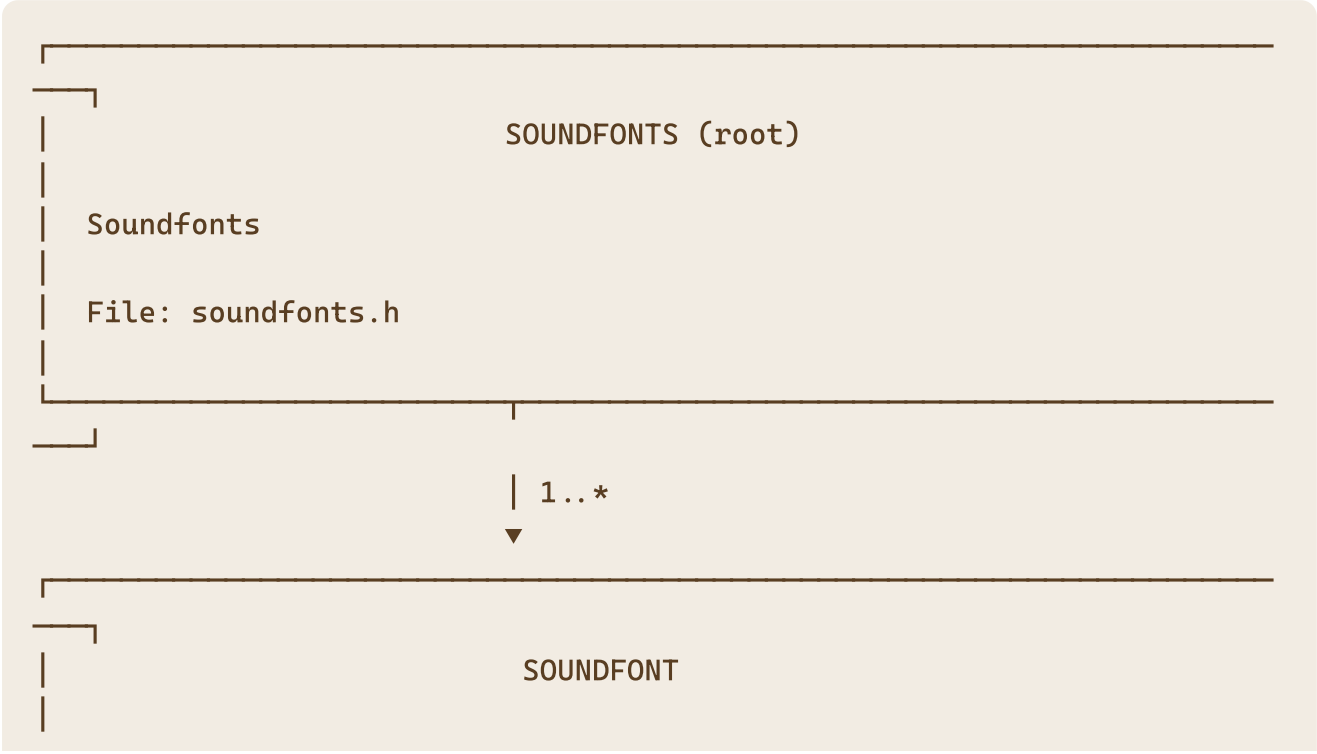
Source File	`sources/sound_engine/voiceparam.cpp`
Function	`VoiceParam::getPosition()`
Lines	328–388
Purpose	Computes the effective sample start, end, loop start, and loop end by applying generator offsets to the sample's base positions
Why it matters	This is how generators `startAddrsOffset` and `startAddrsCoarseOffset` (and equivalents) are actually combined

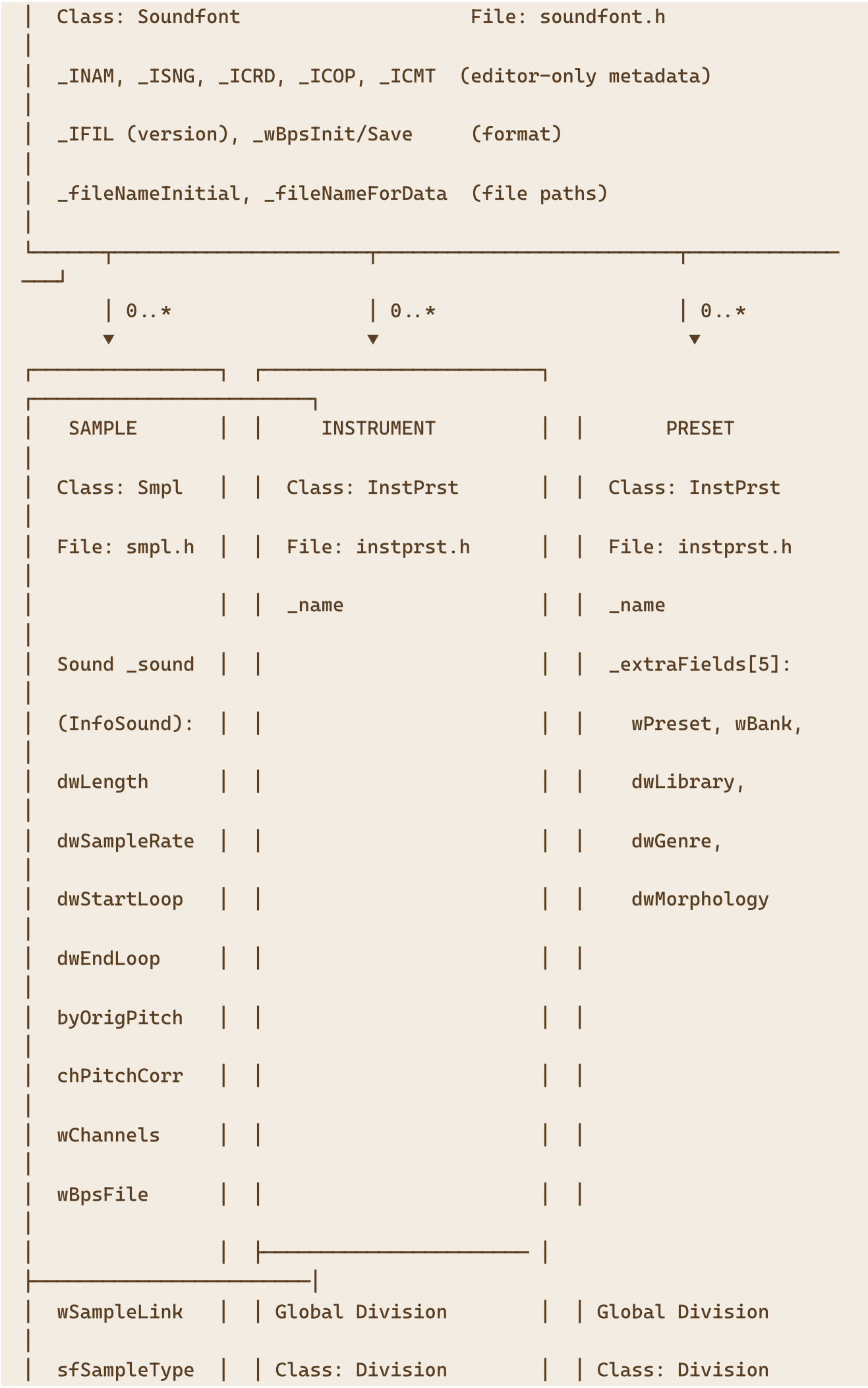
```
case champ_dwStartLoop: {
    qint32 offset =
_parameters[champ_startloopAddrsOffset].getIntValue() +
                    32768 *
_parameters[champ_startloopAddrsCoarseOffset].getIntValue();
    result = static_cast<quint32>(_sampleLoopStart + offset);
    // Clamped to [0, currentLength]
}
```

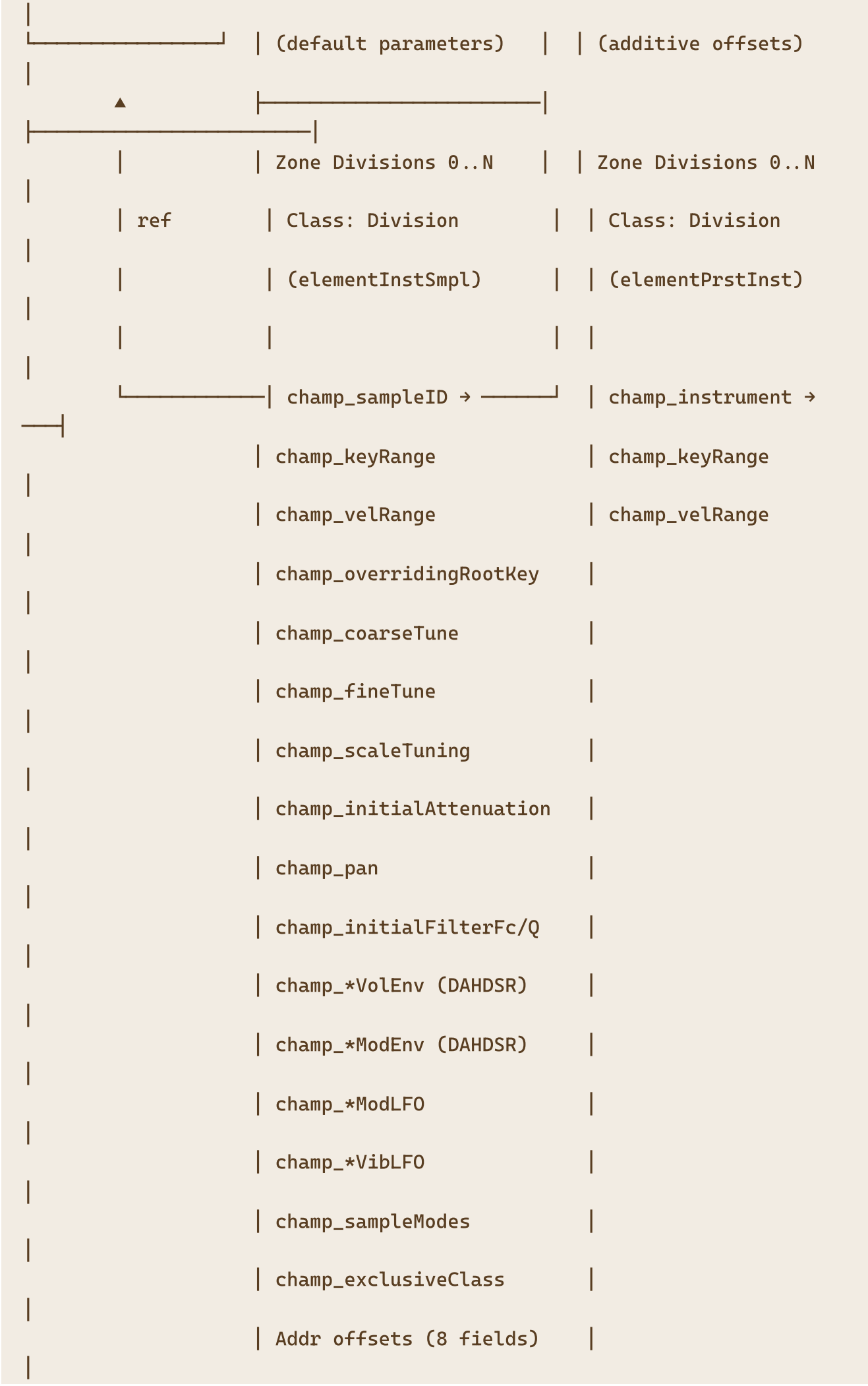
The fine offset (range ±32767) and coarse offset (×32768, range ±32767) are summed. The combined maximum range is ±1,073,709,057 samples — large enough for any practical sample.

---

11. Relationship Diagram




Class: Soundfont
File: soundfont.h
_INAM, _ISNG, _ICRD, _ICOP, _ICMT (editor-only metadata)
_IFIL (version), _wBpsInit/Save (format)
_fileNameInitial, _fileNameForData (file paths)
0..*
0..*
0..*
SAMPLE
Class: Smpl
File: smpl.h
Sound _sound
(InfoSound):
dwLength
dwSampleRate
dwStartLoop
dwEndLoop
byOrigPitch
chPitchCorr
wChannels
wBpsFile
wSampleLink
sfSampleType
INSTRUMENT
Class: InstPrst
File: instprst.h
_name
Global Division
Class: Division
PRESET
Class: InstPrst
File: instprst.h
_name
_extraFields[5]:
wPreset, wBank,
dwLibrary,
dwGenre,
dwMorphology
Global Division
Class: Division


(default parameters)
(additive offsets)
ref
Zone Divisions 0..N
Class: Division
(elementInstSmpl)
Zone Divisions 0..N
Class: Division
(elementPrstInst)
champ_sampleID →
champ_instrument →
champ_keyRange
champ_velRange
champ_overridingRootKey
champ_coarseTune
champ_fineTune
champ_scaleTuning
champ_initialAttenuation
champ_pan
champ_initialFilterFc/Q
champ_*VolEnv (DAHDSR)
champ_*ModEnv (DAHDSR)
champ_*ModLFO
champ_*VibLFO
champ_sampleModes
champ_exclusiveClass
Addr offsets (8 fields)



Data Flow at Note-On

```
MIDI Note-On (key, vel, channel)
        |
        ▼
SoundfontManager.getSoundfonts()
        |
        ▼
Search all Presets:
  For each Preset zone where key ∈ keyRange AND vel ∈ velRange:
        |
        ▼
    Get linked Instrument
        |
        ▼
    Search Instrument zones:
      For each Instrument zone where key ∈ keyRange AND vel ∈ velRange:
            |
            ▼
```

```
        Get linked Sample
            |
            ▼
        VoiceParam::initialize(preset, prstZone, inst, instZone, smpl,
key, vel)
          1. Read sample: byOriginalPitch, chPitchCorrection, length,
loops
          2. Read instrument global division parameters
          3. Read instrument zone parameters (override global)
          4. Read preset global division parameters (additive)
          5. Read preset zone parameters (additive, override prst
global)
          6. Load instrument modulators (global + zone)
          7. Load preset modulators (global + zone)
          8. Apply default SF2 modulators
          9. computeModulations() → final parameter values
            |
            ▼
        Voice created (Module 2 territory)
```

---

12. Embedded Notes Summary

This section collects all embedded notes for easy reference when designing the VHL format.

Sample-Level Notes

Field	ESP32 Needed?	Simplification?
`dwSampleRate`	✅ Yes	None — exact value needed for resampling
`dwLength`	✅ Yes	None
`dwStartLoop`, `dwEndLoop`	✅ Yes	None
`byOriginalPitch`	✅ Yes	None
`chPitchCorrection`	✅ Yes	Could round to integer cents
`sfSampleType` (mono/left/right)	✅ If stereo	Simplify to mono-only for ESP32 to save RAM
`wSampleLink`	✅ If stereo	Simplify to mono-only for ESP32 to save RAM

Field	ESP32 Needed?	Simplification?
Audio data (16-bit)	✅ Yes	Consider 8-bit or 16-bit; 24-bit is not practical on ESP32
Audio data (24-bit extension)	❌ No	Too much RAM; skip
Raw/compressed data	❌ No	ESP32 lacks decompressor RAM
ROM samples	❌ No	Not applicable

Region/Division Level Notes

Feature	ESP32 Needed?	Simplification?
Key range + velocity range	✅ Yes	Required for zone lookup
Sample ID / Instrument ID	✅ Yes	Required for linking
Root key override	✅ Yes	Required for pitch
Coarse/fine/scale tuning	✅ Yes	Required for pitch
Initial attenuation	✅ Yes	Fix the 0.4 scaling error in VHL
Pan	✅ If stereo	Skip if mono-only
Filter cutoff + Q	🔶 Optional	Can implement simple 1-pole LP without Q
Vol env (DAHDSR)	✅ Yes	All 6 stages needed
Mod env (DAHDSR)	🔶 Optional	Simplify to vol-env-only for first version
Vol env key tracking	🔶 Optional	Common musical feature; desirable
Mod LFO	🔶 Optional	Can defer to Module 2
Vib LFO	🔶 Optional	Can defer
Loop mode	✅ Yes	0=no loop, 1=continuous required minimum
Sample address offsets (8 fields)	🔶 Optional	Pre-bake at conversion time; not needed at runtime
Exclusive class	🔶 Optional	Important for percussion (hi-hat choke)
Fixed key / Fixed velocity	🔶 Optional	Simplify: just don't encode it
Chorus/reverb sends	❌ No	No FX bus on ESP32

Feature	ESP32 Needed?	Simplification?
Modulators (custom)	🔶 Optional	Default modulators needed (velocity→volume); custom can be baked
All 10 default SF2 modulators	✅ Partly	#1 (vel→attenuation) is critical; others can be hardcoded

Architecture-Level Notes

Note	Observation
Flatten presets	For VHL, we can pre-bake preset+instrument+global+zone into a single set of zone parameters at conversion time, eliminating runtime layer traversal
No undo/redo	The `ActionManager` / `ActionSet` system is editor-only; ESP32 needs none of it
No tree model	`TreeItem`, `TreeModel`, `TreeItemRoot`, etc. are Qt GUI scaffolding; not needed on ESP32
No SoundfontManager	The singleton pattern and global state are editor conveniences; ESP32 can use direct struct access
Pre-compute default modulations	Default SF2 modulators #1 (vel→atten, concave), #4 (CC1→vibrato), #10 (pitch wheel) can be hardcoded as fixed formulas
Stored vs real values	VHL should store real values directly (Hz, seconds, dB) to avoid the toRealValue conversion at runtime
`champ_endOper = 60`	Only 60 parameters exist at the instrument level. A 60-element array of `float` or `int16` suffices for a zone — very compact
Global division can be baked	When exporting to VHL, merge global division into each zone. Fewer indirection levels at runtime
SFZ import is irrelevant to runtime	SFZ handling is a PC-side parser concern; by the time data reaches ESP32, everything is in SF2-equivalent form

---

End of Module 1 — Instrument Architecture

Next: Module 2 — Playback Engine and Voice Lifecycle